

Tensor Omics (TOX)

Analyze gene expression data and evolutionary relationships between genes

User's Guide

Vivian Bass¹, Anna Beketova¹, Luka Faensen¹, Asis Hallab¹, Aaron Schröder¹, Franz-Eric Sill¹

¹University of Applied Sciences Bingen, Germany

Last revised August 28, 2025

Contents

1	Tensor Omics Method and Theory	3
1.1	The TOX Data Table	3
1.2	From Raw Data to Expression Vectors	3
1.3	Normalization of Expression Values	4
1.4	Calculation of family Centroids	4
1.5	Euclidean Distance Between Expression Vectors	4
1.6	Outlier Detection Based on Family-Scaled Distances	5
1.7	Tissue Versatility and Specificity	5
2	Error Handling	6
3	Implementation Notes	7
3.1	Fortran Pipeline	7
3.2	Python Pipeline	9
3.3	R Pipeline	10

1 Tensor Omics Method and Theory

The Tensor Omics (TOX) tool implements a streamlined pipeline for analyzing gene expression data and evolutionary relationships between genes. The pipeline integrates tools for data normalization, and geometric analysis of gene expression vectors. This document outlines the current implementation details and provides the mathematical formulation of the algorithms used in the prototype.

1.1 The TOX Data Table

The foundational data structure in Tensor Omics is the **Data Table**. Unlike a simple spreadsheet, a Data Table is a conceptual grouping of multiple, type-specific 2D arrays that represent datasets read from a source like a CSV file. Because Fortran is a strongly-typed language, data of different types (e.g., integers, reals, characters) cannot be stored in a single matrix. The TOX library solves this by parsing the source data into a collection of distinct, column-major arrays that share the same number of rows.

The process begins by reading the entire CSV file into a single, general-purpose 2D array of strings. This string array acts as an in-memory staging area. From this staging array, users can then extract specific columns into strongly-typed arrays, creating the components of the Data Table:

- `integer_data(:, :)`: An array for integer columns (e.g., gene IDs).
- `real_data(:, :)`: An array for real-valued columns (e.g., expression levels, scores).
- `logical_data(:, :)`: An array for boolean columns (e.g., status flags).
- `character_data(:, :)`: An array for string columns (e.g., gene names).
- `complex_data(:, :)`: An array for complex number columns.

All downstream analyses in TOX operate on these strongly-typed arrays.

1.2 From Raw Data to Expression Vectors

The starting point for all downstream analyses is a table of gene-wise expression measurements across a set of biological samples, such as tissues, developmental stages, or experimental conditions. These measurements may originate from transcriptomic, proteomic, or metabolomic experiments, typically normalized to account for sequencing depth and technical variation.

What is Gene Expression Data? For each gene, a real-valued quantity is recorded that reflects its biological activity in a given sample. In the case of transcriptomics, this is often the abundance of messenger RNA (mRNA) molecules transcribed from that gene, estimated via sequencing and mapped back to the gene’s known sequence. The result is a non-negative real number proportional to the gene’s transcriptional activity in that sample. After normalization and transformation, these values can be treated as coordinates in a real vector space.

Thus, each gene is represented as a vector $\vec{v} \in \mathbb{R}^n$, where n is the number of biological samples (e.g., tissues or time points), and each component $v_i \geq 0$ corresponds to the expression level in sample i . These vectors encode the distribution of gene activity across conditions and form the geometric basis for all downstream analyses.

Note on Stress Responses. In certain studies, a subset of the samples may correspond to stress conditions. Here, “stress” refers to any external or internal perturbation—such as mechanical damage, heat shock, pathogen infection, or nutrient deprivation—that triggers measurable changes in gene expression. These are often analyzed relative to baseline conditions using log fold-change transformations, such that the resulting vectors reflect differential expression patterns, including up- and downregulation across dimensions.

1.3 Normalization of Expression Values

Raw gene expression data is normalized in the following steps:

1. Division by gene-wise standard deviation for scale normalization.
2. Quantile normalization across samples to reduce global expression biases.
3. Log-transformation: $x \mapsto \log(1 + x)$ to stabilize variance and suppress outliers.
4. For stress response studies, log fold-changes are computed:

$$\Delta_{\text{stress}} = \log(1 + x_{\text{stress}}) - \log(1 + x_{\text{control}})$$

This effectively makes each stress related axis show the fold change in gene expression under stress, e.g. “drought in root divided by control root”.

1.4 Calculation of family Centroids

To summarize the expression profile of each family, Tensor Omics computes a centroid vector for every group. The centroid represents the average expression pattern of all genes assigned to the family across all conditions or tissues.

There are two modes for selecting the set of genes to include in the centroid:

- **Full mode:** All genes assigned to the family are used to compute the centroid (mean vector).
- **Orthologs-only mode:** Only genes explicitly marked as orthologs are included in the centroid calculation. This mode is useful for focusing on conserved gene relationships across species.

1.5 Euclidean Distance Between Expression Vectors

The Euclidean distance is used throughout Tensor Omics to quantify the geometric difference between two gene expression vectors, or between a gene and its family centroid. Given two vectors $\vec{v}_1, \vec{v}_2 \in \mathbb{R}^n$, the Euclidean distance is defined as:

$$d(\vec{v}_1, \vec{v}_2) = \sqrt{\sum_{i=1}^n (v_{1,i} - v_{2,i})^2}$$

This metric measures the straight-line distance between two points in n -dimensional space. In the context of gene expression, it reflects the overall magnitude of difference in expression profiles across all samples or conditions.

For example, the distance between a paralog’s expression vector and the centroid of its gene family is used to detect outlier genes and to quantify the degree of divergence.

1.6 Outlier Detection Based on Family-Scaled Distances

To robustly identify genes with divergent expression profiles within their families, Tensor Omics implements an outlier detection procedure based on family-scaled Euclidean distances. The process consists of the following steps:

1. **Compute Euclidean Distances:** For each gene, calculate the Euclidean distance d_i between its expression vector and the centroid of its assigned family (see Section 1.5).
2. **Estimate Family-Specific Scaling Factors:** For each family f , we compute a family-specific scaling factor. This can be the maximum distance between any ortholog and the ortholog centroid. For families lacking sufficient orthologs, a LOESS smoothing strategy is used to estimate an appropriate scaling factor based on the family’s median intra-member distance.
3. **Relative Distance Index (RDI):** For each gene i in family f , compute the Relative Distance Index (RDI):

$$\text{RDI}_i = \frac{d_i}{d_{scale}}$$

This normalizes the gene’s distance by the expected intra-family variability, making the metric comparable across families of different sizes and variances.

4. **Outlier Detection:** Genes are flagged as outliers if their RDI exceeds a chosen percentile threshold (typically the 95th percentile) of the empirical RDI distribution. That is, gene i is an outlier if $\text{RDI}_i \geq T$, where T is the threshold value corresponding to the desired percentile.

This approach ensures that outlier detection is adaptive to the structure and variability of each gene family, and is robust to differences in family size and dispersion. The LOESS-based scaling is particularly important for families with few members.

1.7 Tissue Versatility and Specificity

Tissue Versatility quantifies how uniformly a gene is expressed across all tissues (or, more generally, axes of the expression space). It is defined as the cosine of the angle between a gene’s expression vector $\vec{v}$ and the space diagonal $\vec{d} = (1, 1, \dots, 1)$. The smaller this angle (the larger the cosine), the greater the tissue (axis) versatility of the gene’s expression.

Interpretation:

- A lower angle (cosine closer to 1) means the gene is more evenly expressed across all tissues (high versatility).
- A larger angle (cosine closer to 0) means the gene is active in only a few tissues (high specificity).

Dimensionality Dependence

The maximum possible angle to the diagonal depends on the number of tissues n . The cosine of this angle is minimal when a gene is expressed in only one tissue:

$$\cos(\varphi_{\max_angle}) = \frac{1}{\sqrt{n}}$$

Normalized Tissue Versatility

To make versatility comparable across datasets with different numbers of tissues, we define a normalized version that scales values into the interval $[0, 1]$:

$$\text{Tissue Versatility} = \frac{\cos(\varphi) - \frac{1}{\sqrt{n}}}{1 - \frac{1}{\sqrt{n}}}$$

This maps the cosine values into the interval $[0, 1]$:

- 1: perfectly uniform expression (maximum versatility)
- 0: expression in only one tissue (minimum versatility)

Tissue Specificity

Tissue specificity is defined as the complement of normalized tissue versatility:

$$\text{Tissue Specificity} = 1 - \text{Tissue Versatility}$$

Values close to 1 indicate high tissue-specificity; values close to 0 indicate broad, uniform expression.

2 Error Handling

In case you encounter errors while using TOX, the following error codes may help you identify the issue. The error codes are grouped into categories based on their nature, such as I/O errors, format validation errors, memory errors, and internal errors.

Table 1: TOX Error Codes Reference

Error Code	Description
0	Success
1xx: I/O / File Reading Errors	
101	Could not open file.
102	Could not read magic number.
103	Could not read array type code.
104	Could not read number of dimensions.
105	Could not read array dimensions.
106	Could not read character length.
107	Could not read array data.
2xx: Format / Input Validation Errors	
200	Invalid format detected (e.g., magic mismatch or corrupt format).
201	Invalid input provided.
202	Empty input arrays provided (e.g., zero-length arrays).
203	Dimension mismatch detected (shape inconsistencies).
204	NaN or Inf found in input data where not allowed.
205	Unsupported data type encountered (e.g., complex types).

Continued on next page

Table 1 – continued from previous page

Error Code	Description
3xx: Memory Errors	
301	Memory allocation failed.
302	Null pointer reference encountered.
5xxx: Fortran Runtime / Unit State Errors	
5002	Fortran runtime error: unit not open or not connected.
9xxx: Internal / Unknown Errors	
9001	Internal error: unexpected state or invariant violated.
9999	Unknown error (fallback for unexpected errors).

3 Implementation Notes

This section details the practical pipeline for using the TOX library to read and process tabular data. The workflow is consistent across Fortran, Python, and R, starting with reading a CSV file into a staging area of strings and then converting columns to their proper data types.

3.1 Fortran Pipeline

In a pure Fortran application, the user follows a two-step process to construct the Data Table.

1. **Read CSV to Strings:** First, call the `read_csv_to_strings` subroutine. This reads the entire CSV file into a 2D allocatable array of characters, which serves as the in-memory source for all subsequent operations.
2. **Extract Typed Columns:** Next, call the specific `read*_columns` subroutines (e.g., `read_integer_columns`, `read_real_columns`) to parse columns from the string array into new, strongly-typed 2D arrays.

Listing 1: Fortran Data Loading Workflow

```

PROGRAM tox_fortran_example
  USE csv_file_reader_module, ONLY: read_csv_to_strings
  USE csv_read_int_module, ONLY: read_integer_columns
  USE csv_read_real_module, ONLY: read_real_columns
  USE csv_parser_module, ONLY: MAX_FIELD_LEN
  IMPLICIT NONE

  CHARACTER(LEN=MAX_FIELD_LEN), ALLOCATABLE :: data_str(:,:),
    header_str(:)
  INTEGER, ALLOCATABLE :: data_int(:,:)
  REAL(8), ALLOCATABLE :: data_real(:,:)
  INTEGER :: status

  CHARACTER(LEN=*), PARAMETER :: FILENAME = 'my_data.csv'

  ! 1. Read entire CSV into a 2D string array
  CALL read_csv_to_strings(FILENAME, .TRUE., ',', header_str,
    data_str, status)
  IF (status /= 0) STOP 'Error reading CSV'

```

```

! 2. Extract specific columns into typed arrays
!   Let's say column 1 is Integer IDs, column 3 is Real scores
CALL read_integer_columns(data_str, [1], data_int, status)
IF (status /= 0) STOP 'Error reading integer columns'

CALL read_real_columns(data_str, [3], data_real, status)
IF (status /= 0) STOP 'Error reading real columns'

! The typed arrays data_int and data_real are now ready for
  analysis.
PRINT *, 'Successfully processed data.'

END PROGRAM tox_fortran_example

```

3. TODO Functions relating section 1.2 - 1.4

4. Calculate Distance to Centroid

For each gene in the expression vector matrix, the function computes the Euclidean distance to its corresponding family centroid and returns a distances array containing the distance value for each expression vector to its family centroid. Call the `distance_to_centroid` function with the following arguments:

Input arguments:

- `n_genes` integer(int32): Total number of genes
- `n_families` integer(int32): Total number of gene families
- `genes` real(real64): Gene expression matrix ($d \times n_genes$)
- `centroids` real(real64): Family centroid matrix ($d \times n_genes$)
- `gene_to_fam` integer(int32): Gene-to-family mapping array with the length of `n_genes` (1-based indexing)
- `d` integer(int32): Expression vector dimension

Output arguments:

- `distances` real(real64): Array of distances for each gene with length of `n_genes`

Listing 2: Distance to Centroid Calculation

```

call distance_to_centroid(n_genes, n_families, genes, centroids,
  gene_to_fam, distances, d)

```

(a) Calculate Euclidean Distance

Alternatively you can calculate a single Euclidean distance between two vectors manually by calling `euclidean_distance` function with the following arguments:

Input arguments:

- `vec1` real(real64): First expression vector as array with length of dimension `d`
- `vec2` real(real64): Second expression vector as array with length of dimension `d`
- `d` integer(int32): Dimension value

Output arguments:

- `result real(real64)`: Euclidean distance value between `vec1` and `vec2`

Listing 3: Euclidean distance calculation

```
call euclidean_distance(vec1, vec2, d, result)
```

5. Detect Gene Outliers

The function first computes family scaling, then calculates relative distance indices (RDI) and identifies outliers based on the specified percentile threshold. It returns a boolean array indicating whether each gene is an outlier. Call the `detect_outliers` function with the following arguments:

Input arguments:

- `n_genes integer(int32)`: Total number of genes
- `n_families integer(int32)`: Total number of gene families
- `distances real(real64)`: Array of distances for each gene to its centroid with length of `n_genes`
- `gene_to_fam integer(int32)`: Gene-to-family mapping array with the length of `n_genes` (1-based indexing)
- **(Optional)** `percentile real(real64)`: Percentile threshold value for outlier detection (default is 95)

Input / Output arguments:

- `work_array real(real64)`: Work array for sorting with length of `n_genes`
- `perm integer(int32)`: Permutation array for sorting with length of `n_genes`
- `stack_left integer(int32)`: Stack array for left indices during sorting with length of `n_genes`
- `stack_right integer(int32)`: Stack array for right indices during sorting with length of `n_genes`
- `loess_x real(real64)`: Reference x-coordinates for loess smoothing with length of `n_families`
- `loess_y real(real64)`: Reference y-coordinates for loess smoothing with length of `n_families`
- `loess_n integer(int32)`: Indices of reference points used for smoothing with length of `n_families`

Output arguments:

- `is_outlier logical`: Boolean array with length of `n_genes` indicating whether each gene is an outlier
- `ierr integer(int32)`: Error code

Listing 4: Gene Outlier Calculation

```
call detect_outliers(n_genes, n_families, distances, gene_to_fam,
                    work_array, perm, stack_left, stack_right, is_outlier, loess_x,
                    loess_y, loess_n, ierr, percentile)
```

Alternatively you can calculate each step of `detect_outliers` manually with the following functions:

(a) Compute Family Scaling

The function calculates a scaling factor for each gene family by computing the median and standard deviation of gene-to-centroid distances within each family, then applies LOESS smoothing to these values. Call the `compute_family_scaling` function with the following arguments:

Input arguments:

- `n_genes` integer(int32): Total number of genes
- `n_families` integer(int32): Total number of gene families
- `distances` real(real64): Array of distances for each gene to its centroid with length of `n_genes`
- `gene_to_fam` integer(int32): Gene-to-family mapping array with the length of `n_genes` (1-based indexing)

Input / Output arguments:

- `loess_x` real(real64): Reference x-coordinates for loess smoothing with length of `n_families`
- `loess_y` real(real64): Reference y-coordinates for loess smoothing with length of `n_families`
- `indices_used` integer(int32): Indices of reference points used for smoothing with length of `n_families`
- `perm_tmp` integer(int32): Permutation array for sorting with length of `n_genes`

Output arguments:

- `dscale` real(real64): Array of scaling factors per family with length of `n_families`
- `family_distances` real(real64): Temporary work array with length of `n_genes` to store distances of genes within each family during calculation
- `ierr` integer(int32): Error code

Listing 5: Compute Family Scaling

```
call compute_family_scaling(n_genes, n_families, distances,
                          gene_to_fam, dscale, loess_x, loess_y, indices_used, perm_tmp,
                          stack_left_tmp, stack_right_tmp, family_distances, ierr)
```

(b) Compute Family Scaling Helper

This helper function allocates internal arrays for easier usage and calls `compute_family_scaling`. Call the `compute_family_scaling_alloc` function with the following arguments:

Input arguments:

- `n_genes` integer(int32): Total number of genes
- `n_families` integer(int32): Total number of gene families

- `distances` `real(real64)`: Array of distances for each gene to its centroid with length of `n_genes`
- `gene_to_fam` `integer(int32)`: Gene-to-family mapping array with the length of `n_genes` (1-based indexing)

Input / Output arguments:

- `loess_x` `real(real64)`: Reference x-coordinates for loess smoothing with length of `n_families`
- `loess_y` `real(real64)`: Reference y-coordinates for loess smoothing with length of `n_families`
- `indices_used` `integer(int32)`: Indices of reference points used for smoothing with length of `n_families`

Output arguments:

- `dscale` `real(real64)`: Array of scaling factors per family with length of `n_families`
- `ierr` `integer(int32)`: Error code

Listing 6: Compute Family Scaling

```
call compute_family_scaling_alloc(n_genes, n_families, distances,
    gene_to_fam, dscale, loess_x, loess_y, indices_used, ierr)
```

(c) **Compute Relative Distance Index (RDI)**

The function calculates the Relative Distance Index (RDI) for each gene by dividing its Eculidean distance to the family centroid by the scaling factor of its family. Call the `compute_rdi` function with the following arguments:

Input arguments:

- `n_genes` `integer(int32)`: Total number of genes
- `distances` `real(real64)`: Array of distances for each gene to its centroid with length of `n_genes`
- `gene_to_fam` `integer(int32)`: Gene-to-family mapping array with the length of `n_genes` (1-based indexing)
- `dscale` `real(real64)`: Array of scaling factors per family with length of `n_families`

Input / Output arguments:

- `sorted_rdi` `real(real64)`: Work array for sorting with length of `n_genes`
- `perm` `integer(int32)`: Pre-initialized (1: `n_genes`) permutation array for sorting with length of `n_genes`
- `stack_left` `integer(int32)`: Stack array for sorting with length of `n_genes`
- `stack_right` `integer(int32)`: Stack array for sorting with length of `n_genes`

Output arguments:

- `rdi` `real(real64)`: Array of RDI values for each gene with length of `n_genes`

Listing 7: Compute Relative Distance Index (RDI)

```
call compute_rdi(n_genes, distances, gene_to_fam, dscale, rdi,
    sorted_rdi, perm, stack_left, stack_right)
```

(d) Identify Outliers

The function identifies outliers based on the top percentile of RDI values. Call the `identify_outliers` function with the following arguments:

Input arguments:

- `n_genes` integer(int32): Total number of genes
- `rdi` real(real64): Array of RDI values for each gene with length of `n_genes`
- `sorted_rdi` real(real64): Array of sorted RDI values for each gene with length of `n_genes` - **this array must be filtered to remove negatives and be sorted in ascending order!**
- **(Optional)** `percentile` real(real64): Percentile threshold value for outlier detection (default is 95)

Output arguments:

- `is_outlier` logical: Boolean array with length of `n_genes` indicating whether each gene is an outlier
- `threshold`: Actual threshold value used for outlier detection

Listing 8: Identify Outliers

```
call identify_outliers(n_genes, rdi, sorted_rdi, is_outlier,
                      threshold, percentile)
```

6. Compute Normalized Tissue Versatility

The function calculates a normalized tissue versatility score for selected gene expression vectors. It is based on the angle between each gene expression vector and the space diagonal. Versatility is normalized to [0, 1], where 0 means uniform expression 1 means expression in only one axis. Call the `compute_tissue_versatility` function with the following parameters:

Input arguments:

- `n_genes` integer(int32): Total number of genes
- `n_vectors` integer(int32): Total number of expression vectors
- `n_selected_axes` integer(int32): Total number of selected axes
- `n_selected_vectors` integer(int32): Total number of selected vectors
- `expression_vectors` real(real64): Expression vector matrix with size `n_axes` × `n_vectors`
- `exp_vecs_selection_index` logical: Logical array with length of `n_vectors` to specify which vectors should be included in the calculation
- `axes_selection` logical: Logical array with length of `n_axes` to specify which axes should be included in the calculation

Output arguments:

- `tissue_versatilities` real(real64): Array with the length of `n_selected_vectors` containing the calculated tissue versatilities
- `tissue_angles_deg` real(real64): Array with the length of `n_selected_vectors` containing the calculated angles in degrees
- `ierr` integer(int32): Error code

Listing 9: Tissue Versatility Calculation

```
call compute_tissue_versatility(n_axes, n_vectors, expression_vectors,
    exp_vecs_selection_index, n_selected_vectors, axes_selection,
    n_selected_axes, tissue_versatilities, tissue_angles_deg, ierr)
```

3.2 Python Pipeline

The Python workflow is exposed through the `tensoromics_functions.py` module. The functions provide a high-level interface that mirrors the Fortran pipeline.

1. **Read CSV to Strings:** Call the `read_csv_as_strings` function. It takes the file path and returns the header and a 2D NumPy array of strings.
2. **Extract Typed Columns:** Pass the string NumPy array to the desired conversion functions, such as `read_integer_columns` or `read_real_columns`, along with a list of 1-based column indices to extract. All returned NumPy arrays are read-only for data safety.

Listing 10: Python Data Loading Workflow

```
import numpy as np
from tensoromics_functions import (
    read_csv_as_strings,
    read_integer_columns,
    read_real_columns
)

# Define the path to the data file
filename = 'my_data.csv'

try:
    # 1. Read the entire CSV into a NumPy array of strings
    header, data_str = read_csv_as_strings(filename)

    print("CSV data read successfully as strings:")
    print(data_str)

    # 2. Extract specific columns into new, typed NumPy arrays
    #     Column 1: Integer IDs
    #     Column 3: Real scores

    integer_data = read_integer_columns(data_str, columns=[1])
    real_data = read_real_columns(data_str, columns=[3])

    print("\nInteger data (column 1):")
    print(integer_data)

    print("\nReal data (column 3):")
    print(real_data)

    # The typed arrays are now ready for analysis (e.g., with SciPy,
    # pandas)

except (FileNotFoundError, ValueError) as e:
    print(f"An error occurred: {e}")
```

3. TODO Functions relating section 1.2 - 1.4

4. Calculate Distance to Centroid

For each gene in the gene expression matrix, the function computes the Euclidean distance to its corresponding family centroid and returns a read-only distances array containing the distance value for each expression vector to its family centroid. Call the `tox_distance_to_centroid` function with the following arguments:

- **genes:** Gene expression matrix array with size of (`n_genes` × `d`)
- **centroids:** Family centroid matrix array with size of (`n_families` × `d`)
- **gene_to_fam:** Gene-to-family mapping array with the length of `n_genes` (0 = no family, >0 = family index)
- **d:** Gene expression matrix dimension as number

Listing 11: Python Distance to Centroid Calculation

```
tox_distance_to_centroid(genes, centroids, gene_to_fam, d)
```

(a) Calculate Euclidean Distance

Alternatively you can calculate a single Euclidean distance between two vectors manually by calling `tox_euclidean_distance` function with the following arguments:

- **vec1:** First vector as array
- **vec2:** Second vector as array

Both vectors must have same length!

Listing 12: Python Euclidean distance calculation

```
tox_euclidean_distance(vec1, vec2)
```

5. Detect Gene Outliers

The function first computes family scaling, then calculates relative distance indices (RDI) and identifies outliers based on the specified percentile threshold. It returns a read-only dictionary containing the outlier boolean values and intermediate results. Call the `tox_detect_outliers` function with the following arguments:

- **distances:** Array of distances for each gene to its centroid
- **gene_to_fam:** Gene-to-family mapping array with the same length as the distances array (0 = no family, >0 = family index)
- **(Optional) percentile:** Percentile threshold value for outlier detection (default is 95)

Return dictionary:

- **outliers:** Boolean array indicating whether each gene is an outlier
- **loess_x:** Array of x coordinates used as reference points for LOESS smoothing (median distance)

- **loess_y**: Array of y coordinates used as reference points for LOESS smoothing (standard deviation)
- **loess_n**: Array with numbers of genes in each family used to compute the corresponding median and standard deviation

Listing 13: Python Gene Outlier Calculation

```
tox_detect_outliers(distances , gene_to_fam , percentile=95.0)
```

Alternatively you can calculate each step of `tox_detect_outliers` manually with the following functions:

(a) Compute Family Scaling

The function calculates a scaling factor for each gene family by computing the median and standard deviation of gene-to-centroid distances within each family, then applies LOESS smoothing to these values. Call the `tox_compute_family_scaling` function with the following arguments:

- **distances**: Array of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping array with the same length as the distances array (0 = no family, >0 = family index)

Return dictionary:

- **dscale**: Array of scaling factors per family
- **loess_x**: Array of x coordinates used as reference points for LOESS smoothing (median distance)
- **loess_y**: Array of y coordinates used as reference points for LOESS smoothing (standard deviation)
- **indices_used**: Array of indices of families that have been included in the calculation

Listing 14: Python Compute Family Scaling

```
tox_compute_family_scaling(distances , gene_to_fam)
```

(b) Compute Family Scaling (Expert Version)

This function is the expert version of `tox_detect_outliers` and requires pre-allocated work arrays for maximum performance and control. It can be used when you need fine-grained control over memory allocation or you are calling this function many times in a tight loop. Call the `tox_compute_family_scaling_expert` function with the following arguments:

- **distances**: Array of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping array with the same length as the distances array (0 = no family, >0 = family index)
- **perm_tmp**: Permutation array for sorting with the same length as the distances array
- **stack_left_tmp**: Stack array for sorting with the same length as the distances array
- **stack_right_tmp**: Stack array for sorting with the same length as the distances array
- **family_distances**: Work array for sorting with the same length as the distances array

Return dictionary:

- **dscale**: Array of scaling factors per family
- **loess_x**: Array of x coordinates used as reference points for LOESS smoothing (median distance)
- **loess_y**: Array of y coordinates used as reference points for LOESS smoothing (standard deviation)
- **indices_used**: Array of indices of families that have been included in the calculation
- **perm_tmp**: Permutation array for sorting
- **stack_left_tmp**: Stack array for sorting
- **stack_right_tmp**: Stack array for sorting
- **family_distances**: Work array for sorting

Listing 15: Python Compute Family Scaling (Expert Version)

```
tox_compute_family_scaling_expert(distances, gene_to_fam, perm_tmp,
                                  stack_left_tmp, stack_right_tmp, family_distances)
```

(c) **Compute Relative Distance Index (RDI)**

The function calculates the Relative Distance Index (RDI) for each gene by dividing its Euclidean distance to the family centroid by the scaling factor of its family. The function returns a numpy array with RDI values for each gene. Call the `tox_compute_rdi` function with the following arguments:

- **distances**: Array of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping array with the same length as the distances array (0 = no family, >0 = family index)
- **dscale**: Array of scaling factors per family

Listing 16: Python Compute Relative Distance Index (RDI)

```
tox_compute_rdi(distances, gene_to_fam, dscale)
```

(d) **Identify Outliers**

The function identifies outliers based on the top percentile of RDI values. Call the `tox_identify_outliers` function with the following arguments:

- **rdi**: Array of RDI values for each gene
- **(Optional) threshold**: Fixed RDI threshold number (if None, percentile value is used)
- **(Optional) percentile**: Percentile threshold number for outlier detection (default is 95)

Return dictionary:

- **outliers**: Boolean array indicating whether each gene is an outlier
- **threshold**: Actual threshold value used for outlier detection

Listing 17: Python Identify Outliers

```
tox_identify_outliers(rdi, threshold=None, percentile=95.0)
```


6. Compute Normalized Tissue Versatility:

The function calculates a normalized tissue versatility score for selected gene expression vectors. It is based on the angle between each gene expression vector and the space diagonal. Versatility is normalized to $[0, 1]$, where 0 means uniform expression 1 means expression in only one axis. Call the `tox_compute_tissue_versatility` function with the following parameters:

- **expression_vectors**: Expression vector matrix array with size `n_axes × n_vectors`
- **vector_selection**: Logical array with length of `n_vectors` to specify which vectors should be included in the calculation
- **axes_selection**: Logical array with length of `n_axes` to specify which axes should be included in the calculation

Return dictionary:

- **tissue_versilities**: Array containing the calculated tissue versilities for selected vectors
- **tissue_angles_deg**: Array containing the calculated angles in degrees for selected vectors
- **n_selected_vectors**: Number of vectors processed
- **n_selected_axes**: Number of axes used in calculation

Listing 18: Python Tissue Versatility Calculation

```
tox_calculate_tissue_versatility(expression_vectors, vector_selection,
                                axis_selection)
```

3.3 R Pipeline

The R pipeline first reads the file into a raw, memory-efficient format (an integer array of ASCII codes) and then performs conversions from that raw format.

1. **Read CSV to Raw ASCII**: Call `tox.read_csv_raw`. This function reads the file and returns a list containing the data as a 3D integer array of ASCII codes. This intermediate format is efficient for passing between functions.
2. **Extract Typed Columns**: Pass the raw data array (`$data`) from the previous step to conversion functions like `tox.read_integer_columns` to get typed matrices.
3. **(Optional) Convert to Strings**: For inspection, you can convert the raw ASCII data to a human-readable character matrix using `tox.convert_ascii_to_strings`.

Listing 19: R Data Loading Workflow

```
# Load the TOX API functions
source("R/tensoromics_functions.R")

filename <- "my_data.csv"

# 1. Read the CSV file into a raw ASCII integer array format
```

```

raw_data_list <- tox.read_csv_raw(filename)

# For user inspection, convert the raw data to a character matrix
string_matrix <- tox.convert_ascii_to_strings(raw_data_list$data)
cat("Data as character matrix:\n")
print(string_matrix)

# 2. Use the raw_data_list$data to extract typed columns
#   Column 1: Integer IDs
#   Column 3: Real scores
integer_matrix <- tox.read_integer_columns(raw_data_list$data, columns
    = 1)
real_matrix <- tox.read_real_columns(raw_data_list$data, columns = 3)

cat("\nExtracted integer data (column 1):\n")
print(integer_matrix)

cat("\nExtracted real data (column 3):\n")
print(real_matrix)

```

4. TODO Functions relating section 1.2 - 1.4

5. Calculate Distance to Centroid

For each gene in the gene expression matrix, the function computes the Euclidean distance to its corresponding family centroid and returns a numeric vector of distances from each gene to its family centroid. Call the `tox_distance_to_centroid` function with the following arguments:

- **genes:** Gene expression matrix with size of (`n_genes` × `d`)
- **centroids:** Family centroid matrix with size of (`n_families` × `d`)
- **gene_to_fam:** Gene-to-family mapping integer vector with the length of `n_genes` (0 = no family, >0 = family index)
- **d:** Gene expression matrix dimension as integer

Listing 20: R Distance to Centroid Calculation

```
tox_distance_to_centroid(genes, centroids, gene_to_fam, d)
```

(a) Calculate Euclidean Distance

Alternatively you can calculate a single Euclidean distance between two vectors manually by calling `tox_euclidean_distance` function with the following arguments:

- **vec1:** First vector
- **vec2:** Second vector

Both vectors must have same length!

Listing 21: R Euclidean distance calculation

```
tox_euclidean_distance(vec1, vec2)
```

6. Detect Gene Outliers

The function first computes family scaling, then calculates relative distance indices (RDI) and identifies outliers based on the specified percentile threshold. It returns a list of components containing the outlier boolean values and intermediate results. Call the `tox_detect_outliers` function with the following arguments:

- **distances**: Numeric vector of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping integer vector mapping with the same length as the distances vector (0 = no family, >0 = family index)
- **n_families**: Integer number of families
- **(Optional) percentile**: Percentile threshold value for outlier detection (default is 95.0)

Return list:

- **is_outlier**: Logical vector indicating whether each gene is an outlier
- **loess_x**: Numeric vector of x coordinates used as reference points for LOESS smoothing (median distance)
- **loess_y**: Numeric vector of y coordinates used as reference points for LOESS smoothing (standard deviation)
- **loess_n**: Integer vector with numbers of genes in each family used to compute the corresponding median and standard deviation

Listing 22: R Gene Outlier Calculation

```
tox_detect_outliers(distances, gene_to_fam, n_families, percentile)
```

Alternatively you can calculate each step of `tox_detect_outliers` manually with the following functions:

(a) Compute Family Scaling

The function calculates a scaling factor for each gene family by computing the median and standard deviation of gene-to-centroid distances within each family, then applies LOESS smoothing to these values. Call the `tox_compute_family_scaling` function with the following arguments:

- **distances**: Numeric vector of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping integer vector mapping with the same length as the distances vector (0 = no family, >0 = family index)
- **n_families**: Integer number of families

Return dictionary:

- **dscale**: Numeric vector of scaling factors per family
- **loess_x**: Numeric vector of x coordinates used as reference points for LOESS smoothing (median distance)
- **loess_y**: Numeric vector of y coordinates used as reference points for LOESS smoothing (standard deviation)
- **indices_used**: Integer vector of indices of families that have been included in the calculation

Listing 23: R Compute Family Scaling

```
tox_compute_family_scaling(distances, gene_to_fam, n_families)
```

(b) **Compute Family Scaling (Expert Version)**

This function is the expert version of `tox_detect_outliers` and requires pre-allocated work arrays for maximum performance and control. It can be used when you need fine-grained control over memory allocation or you are calling this function many times in a tight loop. Call the `tox_compute_family_scaling_expert` function with the following arguments:

- **distances**: Numeric vector of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping integer vector mapping with the same length as the distances vector (0 = no family, >0 = family index)
- **n_families**: Integer number of families
- **perm_tmp**: Permutation integer vector for sorting with the same length as distances
- **stack_left_tmp**: Stack integer vector for sorting with the same length as distances
- **stack_right_tmp**: Stack integer vector for sorting with the same length as distances
- **family_distances**: Work integer vector for sorting with the same length as distances

Return list:

- **dscale**: Numeric vector of scaling factors per family
- **loess_x**: Numeric vector of x coordinates used as reference points for LOESS smoothing (median distance)
- **loess_y**: Numeric vector of y coordinates used as reference points for LOESS smoothing (standard deviation)
- **indices_used**: Integer vector of indices of families that have been included in the calculation
- **perm_tmp**: Permutation vector for sorting
- **stack_left_tmp**: Stack vector for sorting
- **stack_right_tmp**: Stack vector for sorting
- **family_distances**: Work vector for sorting

Listing 24: R Compute Family Scaling (Expert Version)

```
tox_compute_family_scaling_expert(distances, gene_to_fam, n_families, perm_tmp, stack_left_tmp, stack_right_tmp, family_distances)
```

(c) **Compute Relative Distance Index (RDI)**

The function calculates the Relative Distance Index (RDI) for each gene by dividing its Euclidean distance to the family centroid by the scaling factor of its family. Call the `tox_compute_rdi` function with the following arguments:

- **distances**: Numeric vector of distances for each gene to its centroid
- **gene_to_fam**: Gene-to-family mapping integer vector mapping with the same length as the distances vector (0 = no family, >0 = family index)
- **dscale**: Numeric vector of scaling factors per family

Return list:

- **rdi**: Numeric vector containing the RDI value for each gene
- **sorted_rdi**: Numeric vector containing the RDI values sorted in ascending order

Listing 25: R Compute Relative Distance Index (RDI)

```
tox_compute_rdi(distances, gene_to_fam, dscale)
```

(d) **Identify Outliers**

The function identifies outliers based on the top percentile of RDI values. Call the `tox_identify_outliers` function with the following arguments:

- **rdi**: Numeric vector containing the RDI value for each gene
- **(Optional) percentile**: Percentile threshold number for outlier detection (default is 95.0)

Return list:

- **is_outlier**: Logical vector indicating whether each gene is an outlier
- **threshold**: Actual threshold value used for outlier detection

Listing 26: R Identify Outliers

```
tox_identify_outliers(rdi, percentile)
```

7. Compute Normalized Tissue Versatility:

The function calculates a normalized tissue versatility score for selected gene expression vectors. It is based on the angle between each gene expression vector and the space diagonal. Versatility is normalized to $[0, 1]$, where 0 means uniform expression 1 means expression in only one axis. Call the `tox_compute_tissue_versatility` function with the following parameters:

- **expression_vectors**: Numeric matrix array with size `n_axes × n_vectors`
- **vector_selection**: Logical vector with length of `n_vectors` to specify which vectors should be included in the calculation
- **axes_selection**: Logical vector with length of `n_axes` to specify which axes should be included in the calculation

Return list:

- **tissue_versatilities**: Numeric vector containing the calculated tissue versatilities for selected vectors
- **tissue_angles_deg**: Numeric vector containing the calculated angles in degrees for selected vectors
- **n_selected_vectors**: Integer number of vectors processed
- **n_selected_axes**: Integer number of axes used in calculation

Listing 27: R Tissue Versatility Calculation

```
tox_calculate_tissue_versatility(expression_vectors, vector_selection,  
axis_selection)
```